Kubernetes

Kubernetes (K8s) is an **open-source container orchestration platform** used to **automatically deploy, manage, and scale containerized applications**.

In simple words:
Kubernetes helps you run many containers (like Docker containers) in a **smart and automated way** across multiple machines.

Why Kubernetes?

Before Kubernetes, managing containers manually was difficult.

Problems without Kubernetes:

- Hard to manage many containers
- No automatic scaling
- No self-healing if containers fail
- Difficult networking between containers

Kubernetes solves:

- **Auto Scaling** → increases/decreases containers based on load
- **Auto Healing** → restarts failed containers automatically
- **Load Balancing** → distributes traffic
- **Service Discovery** → containers find each other easily
- **Rolling Updates** → update apps without downtime

Kubernetes Architecture

Kubernetes works in a **cluster**.

Cluster = Master Node + Worker Nodes

1)Control Plane (Master Node)

This manages the cluster.

Main components:

- **API Server** → Entry point (kubectl talks to this)
- **Scheduler** → Decides where pods run
- **Controller Manager** → Maintains desired state
- **etcd** → Stores cluster data (database)

2) Worker Node(Data plane)

This runs actual applications.

Components:

- **Kubelet** → Communicates with master
- **Kube Proxy** → Handles networking
- **Container Runtime** → Runs containers (Docker / containerd)

Kubernetes Hierarchy (Flow)

Your understanding is almost correct. Here is the correct flow:

```
Cluster
   ↓
Node
   ↓
Pod
   ↓
Container
```

Explanation:

- **Cluster** → Whole Kubernetes system
- **Node** → Machine (VM or physical)
- **Pod** → Smallest unit in Kubernetes
- **Container** → Application running inside pod

Important:
A **Pod can contain one or more containers**.

Kubernetes Components

Pod

- Smallest unit
- Contains container(s)
- Has its own IP

Deployment

- Manages pods
- Ensures required number of pods are running

Service

- Exposes pods to network
- Provides stable IP

Namespace

- Logical separation of resources

ConfigMap & Secret

- Store configuration data

Uses of Kubernetes

Kubernetes is used for:

- Running microservices architecture
- Deploying cloud applications
- CI/CD pipelines
- Scaling applications automatically
- Managing large distributed systems

Pod Creation (Commands)

Step 1: Create a pod directly

```
kubectl run mypod --image=nginx
```

👉 This creates a pod with nginx image

Step 2: Check pods

```
kubectl get pods
```

Step 3: Describe pod

```
kubectl describe pod mypod
```

Step 4: Delete pod

```
kubectl delete pod mypod
```

8. Pod Creation using YAML File

Example: pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: mypod
  labels:
    app: nginx
spec:
  containers:
  - name: nginx-container
    image: nginx
    ports:
    - containerPort: 80
```

Apply YAML file

```
kubectl apply -f pod.yaml
```

Delete YAML resources

```
kubectl delete -f pod.yaml
```

Important Concepts

Auto Scaling

Automatically increases/decreases pods based on traffic.

Auto Healing

If a pod crashes → Kubernetes restarts it.

Ingress

- Manages external HTTP/HTTPS access
- Acts like a **router**

Real-Time Example

Imagine you have a website:

- 100 users → 2 pods enough
- 10,000 users → Kubernetes creates more pods
- If a pod crashes → Kubernetes creates a new one

Everything happens automatically.

Summary

- Kubernetes = Container manager
- Pod = Smallest unit
- Deployment = Manages pods
- Service = Exposes pods
- Node = Machine
- Cluster = Group of nodes